

CallVerse — Workforce Manager (Reinforcement Learning)

Lot RL : environnement Gymnasium, PPO, baselines, protocole expérimental

Phase 1 — Fondations et contrats

Mission du lot, et pourquoi tu démarres en premier

Ce que tu construis

1. L'environnement de simulation (couche 1 du Client Simulator)
2. Les baselines (à faire AVANT le RL)
3. L'agent RL

Sur l'espace d'action

Sur la fonction de récompense

Protocole expérimental

Métriques

Architecture d'exécution

Ne rien persister pendant l'entraînement

Plan de travail — semaine 1

Pièges à éviter

Plan de repli

Critères de sortie du lot

Mission du lot, et pourquoi tu démarres en premier

Le Workforce Manager décide de l'allocation des ressources du centre : réaffecter des conseillers entre compétences, ajuster les priorités, gérer les pics. Il apprend par renforcement dans le Digital Twin.

Ce lot est le chemin critique du projet côté ESPRIT, et il est totalement indépendant. L'environnement Gymnasium n'a besoin ni de Spring Boot, ni de PostgreSQL, ni d'Angular : c'est un simulateur de file d'attente en Python pur.

Tu démarres le jour 1 sans attendre personne, et tu dois avoir une **baseline chiffrée en fin de semaine 2**. Si le RL n'est attaqué qu'en phase 4, il n'aura pas le temps de converger et le projet perd sa brique Machine Learning la plus solide.

Ce que tu construis

1. L'environnement de simulation (couche 1 du Client Simulator)

C'est ton environnement RL, et c'est aussi le générateur d'arrivées du Digital Twin. Il décide **qui arrive et quand** :

```
Profil de charge : loi de Poisson, lambda variable selon l'heure
08h-10h : lambda = 12/min (pic matinal)
10h-12h : lambda = 8/min
12h-14h : lambda = 4/min (creux déjeuner)
...
+ événements injectés : incident réseau à t=120 -> lambda x4 pendant 30 min

Pour chaque arrivée :
- tire un client (profil comportemental, historique, contrat)
- tire un motif : facturation 35%, technique 40%,
                  commercial 15%, résiliation 10%
- tire un profil : calme 40%, impatient 25%, insatisfait 20%,
                  exigeant 10%, à risque de churn 5%
```

Cette couche est **purement paramétrique, sans LLM**. C'est ce qui la rend gratuite et rapide, et donc capable d'alimenter des dizaines de milliers de pas d'entraînement.

Le modèle du centre : files par compétence, conseillers avec compétences et niveaux, durée de traitement tirée d'une distribution, patience des clients qui décroît, abandon quand elle est épuisée.

2. Les baselines (à faire AVANT le RL)

Deux stratégies de référence :

- **STATIC_FIFO** — affectation statique par compétence, premier arrivé premier servi. La baseline simple.

- **THRESHOLD** — heuristique à seuil : « si la file d’une compétence dépasse N, réaffecter un conseiller disponible d’une autre compétence ». **C’est la baseline exigeante**, et c’est celle que le jury demandera.

Les faire en premier est une décision de gestion du risque : si le RL ne converge pas, tu as quand même des résultats à présenter.

3. L’agent RL

Élément	Contenu
État	Longueurs de files par compétence, temps d’attente courants, conseillers disponibles par compétence, SLA cumulé, heure, tendance des arrivées
Action	Espace discret restreint : réaffectation entre compétences, ajustement de priorité, ouverture de créneau
Récompense	Combinaison pondérée : SLA respecté, temps d’attente, abandons, occupation, coût
Algorithme	PPO via Stable-Baselines3
Environnement	Gymnasium écrit sur mesure

Sur l’espace d’action

Garde-le petit. Une dizaine d’actions discrètes maximum. Un espace trop riche et le PPO ne converge pas dans le temps imparti. C’est le premier facteur d’échec sur ce type de projet.

Sur la fonction de récompense

Les pondérations doivent être **justifiées et soumises à une analyse de sensibilité** : tu montres ce qui se passe quand tu privilégies le SLA plutôt que le coût. C’est un résultat scientifique en soi, et ça répond à la question « d’où sortent vos poids ? » que le jury posera.

Protocole expérimental

C’est ce qui transforme ton code en preuves. À respecter strictement.

- **Baselines** : `STATIC_FIFO` et `THRESHOLD`.
- **Conditions** : charge faible / moyenne / saturée, plus un scénario de pic d’incident et un scénario de sous-effectif.
- **Réplifications** : minimum **5 seeds** par cellule (condition x stratégie). Résultats rapportés en moyenne plus ou moins écart-type. **Un run isolé n’est jamais présenté comme un résultat** — le jury demandera l’écart-type.
- **Contrôle** : à seed identique, le flux d’arrivées et les profils sont strictement identiques entre stratégies. Le delta mesure l’effet de la stratégie et rien d’autre.

Matrice complète : 3 charges x 4 stratégies x 5 seeds = **60 runs d’évaluation**.

Métriques

Métrique	Définition
Temps d’attente moyen	Moyenne de <code>wait_seconds</code>
P95 d’attente	95e percentile, révèle les cas extrêmes
SLA	Part des demandes traitées sous le seuil (ex. 80% en moins de 60s)
Taux d’abandon	Part des clients partis avant prise en charge
Occupation	Part du temps où les conseillers sont occupés
Coût	Estimation basée sur l’effectif mobilisé
Équité	Ratio attente de la pire compétence sur attente moyenne

Architecture d’exécution

L’entraînement se fait hors ligne, sur des milliers d’épisodes accélérés. En exploitation, on charge la policy entraînée et on l’interroge à chaque décision, ce qui prend quelques millisecondes.

```

POST /ai/workforce/decide
{
  "queues": {"TECHNICAL": 34, "BILLING": 3, "COMMERCIAL": 1},
  "avg_wait": {"TECHNICAL": 260, "BILLING": 40, "COMMERCIAL": 15},
  "available": {"TECHNICAL": 0, "BILLING": 3, "COMMERCIAL": 2},
  "sla_today": 0.71,
  "hour": 11,
  "trend": 1.9
}
-> {
  "action": {"type": "REASSIGN", "from": "BILLING",
            "to": "TECHNICAL", "count": 2},
  "reason": "string",
  "expected_gain": {"wait_seconds": -140}
}

```

Le champ `reason` alimente la table `agent_decision` et le panneau XAI de la supervision. Il peut être généré par un LLM, **mais le LLM ne prend jamais la décision** : il formule la justification d'une décision déjà prise par la policy.

Ne rien persister pendant l'entraînement

Des milliers d'épisodes x des milliers de pas produiraient des millions de lignes inutiles. **Seuls les 60 runs d'évaluation finale vont en base.** L'entraînement écrit ses courbes dans TensorBoard ou un simple CSV local.

Plan de travail – semaine 1

Jour	Tâche
1	Modèle du centre en Python : files, conseillers, compétences, durées
2	Générateur d'arrivées paramétrique avec profils de charge
3	Environnement Gymnasium : <code>reset()</code> , <code>step()</code> , espaces définis
4	Baseline <code>STATIC_FIFO</code> exécutable, KPI calculés
5	Baseline <code>THRESHOLD</code> , premiers chiffres comparatifs

Objectif de fin de semaine 2 : les deux baselines tournent sur les 3 niveaux de charge, 5 seeds chacune, et les KPI sont produits. Tu peux répondre chiffres à l'appui à « combien attend un client en charge moyenne avec l'allocation statique ? »

Pièges à éviter

1. **Commencer par le RL au lieu des baselines.** Sans point de comparaison, tes résultats ne valent rien.
2. **Espace d'action trop grand.** Premier facteur de non-convergence.
3. **Appeler le LLM dans la boucle d'entraînement.** Coût et durée prohibitifs.
4. **Un seul run par configuration.** Le jury demandera l'écart-type.
5. **Persister les données d'entraînement.** Des millions de lignes pour rien.
6. **Attendre le backend ou la base.** Ton lot est autonome, en Python pur.

Plan de repli

Si le PPO ne converge pas malgré les ajustements, tu bascules sur une heuristique optimisée (seuils calibrés par recherche sur grille) et tu **documentes l'échec du RL avec ses causes**. Un échec analysé est un résultat scientifique valable et sera mieux noté qu'un résultat inventé.

Critères de sortie du lot

- ☐ Environnement Gymnasium instanciable, un épisode aléatoire tourne
- ☐ Espaces d'observation et d'action documentés
- ☐ Générateur d'arrivées avec 3 profils de charge et injection d'événements
- ☐ Baseline `STATIC_FIFO` exécutable et ses KPI calculés
- ☐ Baseline `THRESHOLD` exécutable et ses KPI calculés
- ☐ Reproductibilité vérifiée : même seed, mêmes résultats